

LECTURE 12

딥러닝

경북대학교

전자공학부

김현수

목차

14.1 딥러닝의 개요

14.2 딥러닝 분류

14.3 딥러닝 회귀

14.4 [플러스 예제] 당뇨병 진행도 예측

학습목표

- 딥러닝의 개념을 설명할 수 있습니다.
- 딥러닝 절차를 설명할 수 있습니다.
- 딥러닝 분류 프로그램을 작성할 수 있습니다.
- 딥러닝 회귀 프로그램을 작성할 수 있습니다.

딥러닝의 개요

1. 딥러닝의 개요

■ 신경망과 딥러닝

- 신경망(neural network) 은 생물학적 신경계의 작용을 모방한 머신러닝 기법의 일종
- 인간의 신경계를 구성하는 기본 단위는 신경 세포인 '뉴런'이며, 뉴런은 자극을 받아들이고 신호를 전달하는 역할
- 뉴런에는 핵과 세포질로 이루어진 신경 세포체와 이곳에서 뻗어나온 가지 돌기 및 축삭 돌기가 있음
 - 가지 돌기는 다른 뉴런이나 세포에서 오는 신호를 받아들이고, 축삭 돌기는 다른 뉴런이나 세포로 신호를 전달
 - 가지 돌기를 통해 수집한 신호가 결합해서 일정 수준(임계값)을 넘으면 핵에서 신호를 적당히 처리
 - 그 다음 축삭 돌기 말단을 통해 다른 뉴런의 가지 돌기로 신호를 전달

1. 딥러닝의 개요

■ 신경망과 딥러닝

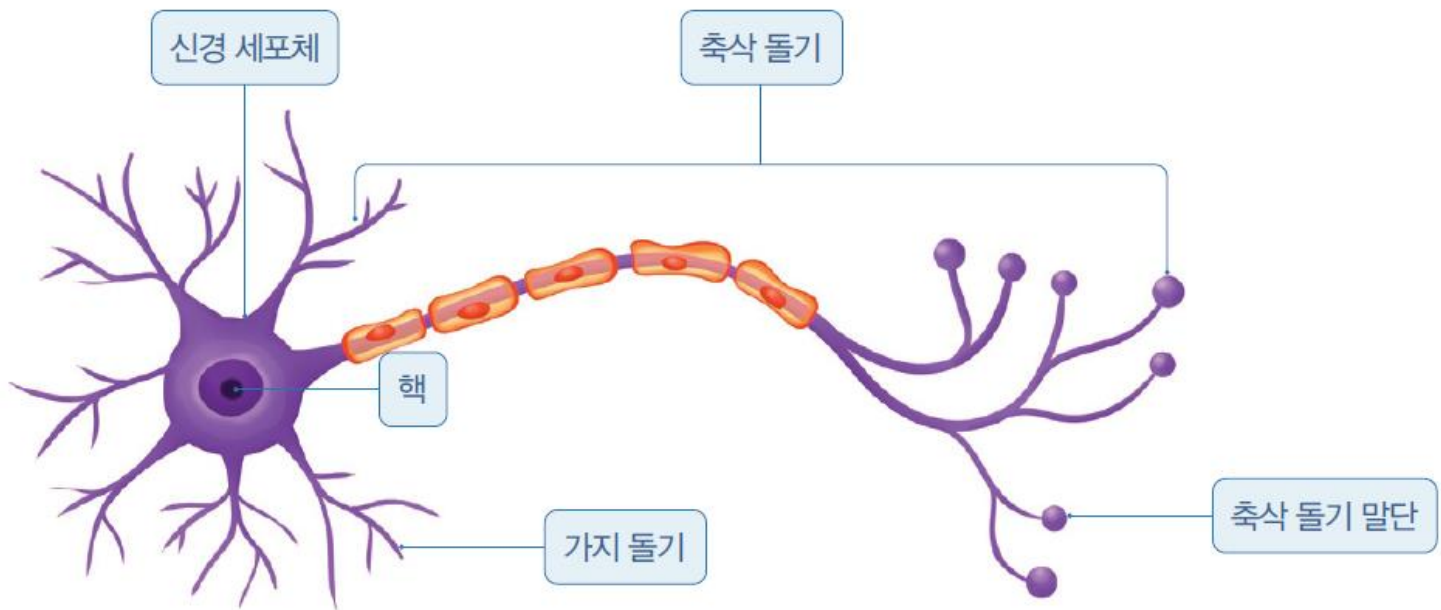


그림 14-1 생물학적 뉴런

1. 딥러닝의 개요

■ 신경망과 딥러닝

- 신경망에도 생물학적 뉴런과 유사한 역할을 하는 '인공 뉴런'이 있음
- 인공 뉴런은 여러 입력 값이 들어왔을 때 입력 값마다 정해진 가중치를 곱한 후에 모두 더한 값이 일정한 값을 넘으면 출력

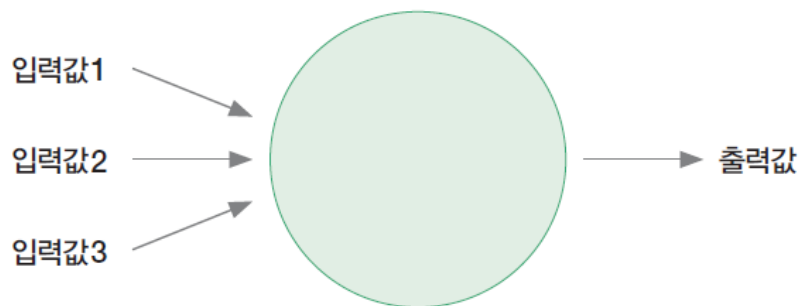


그림 14-2 인공 뉴런

1. 딥러닝의 개요

■ 신경망과 딥러닝

- [그림 14-3]은 뉴런이 결합한 구조
- 뉴런 하나가 여러 개의 뉴런으로부터 신호를 받아 다시 여러 개의 뉴런으로 신호를 전달

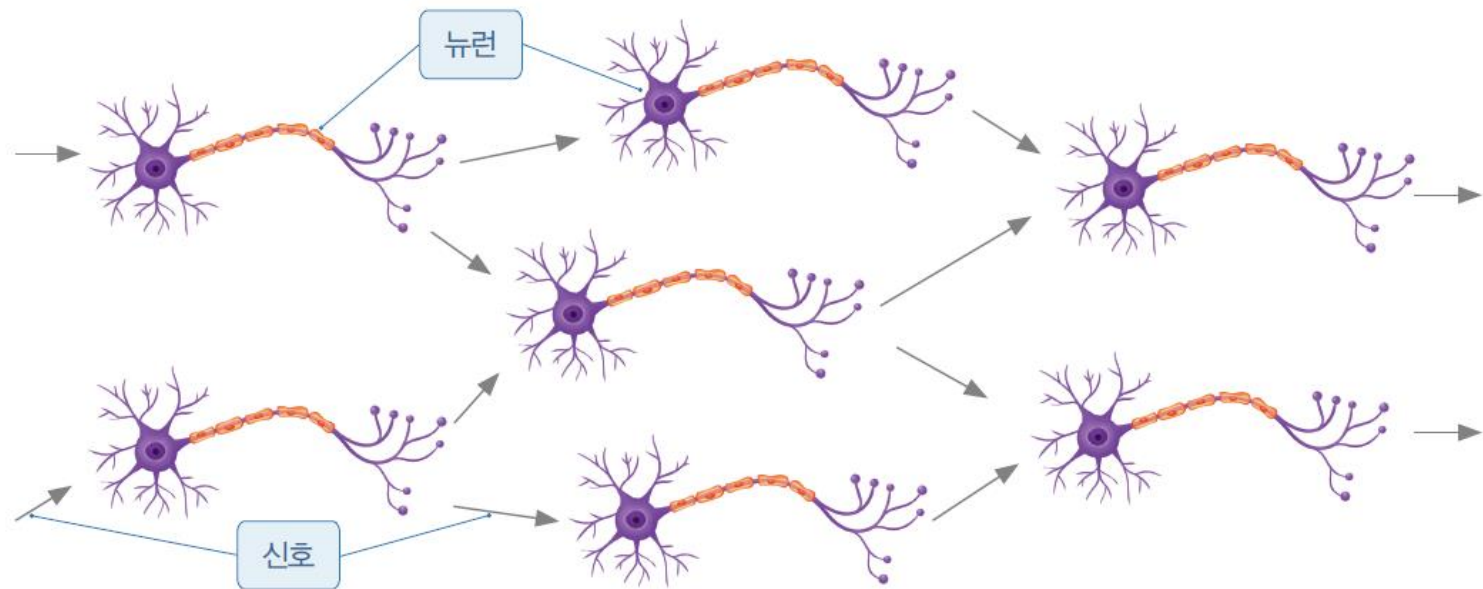


그림 14-3 상호 연결된 생물학적 뉴런

1. 딥러닝의 개요

■ 신경망과 딥러닝

- 신경망 역시 인공 뉴런이 연속된 층으로 연결되어 있음
- 각 층의 인공 뉴런은 다음 층의 인공 뉴런과 연결되어 있음
- 입력값이 들어가는 층을 입력층이라 하고, 입력층과 출력층 사이의 층을 은닉층, 출력값이 나오는 층을 출력층이라 함
- 인공 뉴런 대신 노드라는 용어를 사용

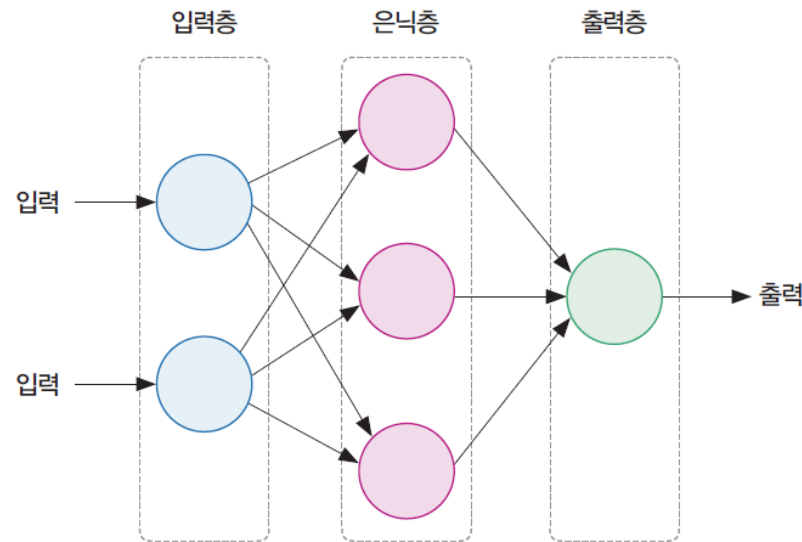


그림 14-4 신경망

1. 딥러닝의 개요

■ 신경망과 딥러닝

- 딥러닝(deep learning): 신경망을 기반으로 하는 머신러닝의 한 분야로, 신경망의 계층을 여러 개로 늘려 복잡한 패턴을 학습하도록 하는 기술

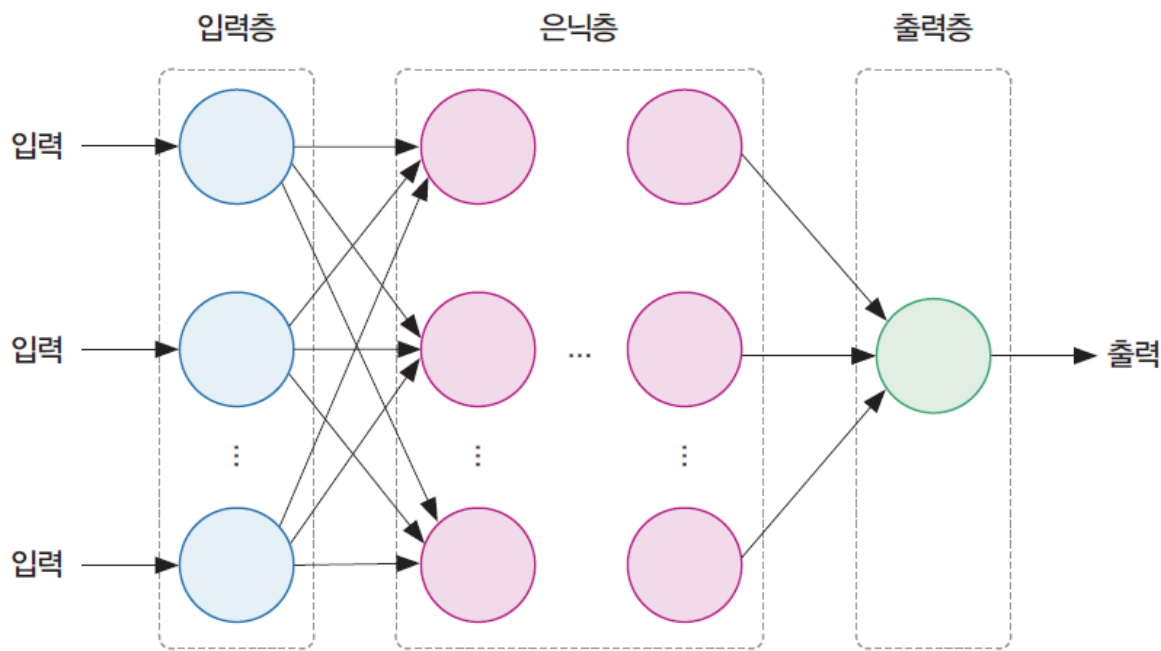


그림 14-5 딥러닝의 구조

1. 딥러닝의 개요

■ 딥러닝의 절차

- 딥러닝의 전체적인 절차는 13장에서 살펴본 머신러닝의 절차와 동일한데, 모델 생성 단계를 [그림 14-6]과 같이 세분화할 수 있음

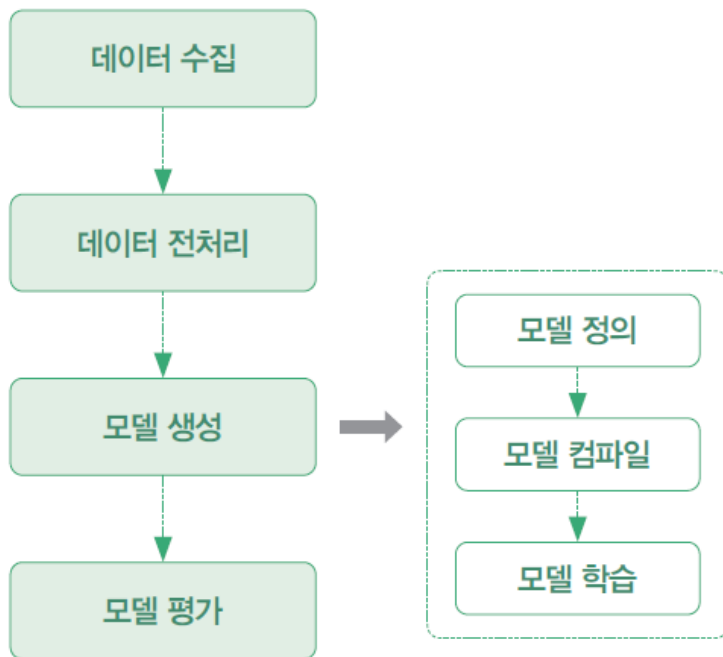


그림 14-6 딥러닝의 절차

딥러닝 분류

2. 딥러닝 분류

■ 당뇨병 발병 예측하기

- 당뇨병 발병을 예측하는 사례를 통해 딥러닝 모델 개발에 대해 살펴보기
- 대표적인 딥러닝 라이브러리인 케라스(keras) 를 사용해서 딥러닝 모델을 만들어보자
- 캐글(Kaggle)에서 제공하는 피마 인디언(Pima Indians) 의 당뇨병 데이터 세트를 활용



그림 14-7 피마 인디언 © 브리태니커

2. 딥러닝 분류

■ 당뇨병 발병 예측하기

- 데이터 세트는 캐글 웹사이트(<https://www.kaggle.com/datasets/uciml/pima-indians-diabetes-database>)에서 수집할 수 있음
- 피마 인디언 혈통의 21세 이상 여성 환자에게 당뇨병이 있는지를 측정한 결과
- 환자 768명의 임신 횟수, 포도당 수치, 혈압, 피부 두께, 인슐린, BMI, 당뇨병 가족력, 나이, 당뇨병 여부로 이루어짐
- 임신 횟수, 포도당 수치, 혈압, 피부 두께, 나이 등 8개 속성이 특성
- 당뇨병 여부는 타깃에 해당

표 14-1 당뇨병 데이터 세트

임신 횟수	포도당 수치	혈압	피부 두께	...	나이	당뇨병 여부
6	148	72	35	...	50	1
1	85	66	29	...	31	0

2. 딥러닝 분류

■ 당뇨병 발병 예측하기

■ 데이터 수집하기

- 캐글에서 데이터 세트를 다운로드하거나 이 책의 부록으로 제공하는 파일을 준비
- 코랩에서 데이터 세트 'diabetes.csv'를 불러와서 df라는 이름으로 저장

코드 14-1

```
01 import pandas as pd
02
03 df = pd.read_csv('diabetes.csv')
```

2. 딥러닝 분류

■ 당뇨병 발병 예측하기

■ 데이터 수집하기

- df의 크기를 출력하고 데이터프레임의 처음 부분을 확인

코드 14-2

```
01 print(len(df))  
02 display(df.head())
```

768

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1

2. 딥러닝 분류

■ 당뇨병 발병 예측하기

■ 데이터 전처리하기

- [코드 14-2]의 실행 결과에서 피부 두께인 SkinThickness 속성과 인슐린인 Insulin 속성은 0이 될 수 없음에도 df에서 0 값이 보임
- 이런 값은 딥러닝 모델의 정확도를 떨어뜨리므로 0이 포함된 행을 삭제
- 0이 되어서는 안 되는 포도당 수치, 혈압, 피부 두께, 인슐린, BMI 속성 이름을 columns 리스트에 저장
- replace 함수를 이용해서 이 열의 값이 0일 때 값을 NaN으로 교체

코드 14-3

```
01 import numpy as np
02
03 columns = ['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI']
04 for col in columns:
05     df[col].replace(0, np.nan, inplace=True)
```

2. 딥러닝 분류

■ 당뇨병 발병 예측하기

■ 데이터 전처리하기

- dropna 함수를 이용해서 NaN을 포함한 행을 삭제

코드 14-4

```
01 df.dropna(inplace=True)
02 print(len(df))
```

392

2. 딥러닝 분류

■ 당뇨병 발병 예측하기

■ 데이터 전처리하기

- 데이터 세트를 특성과 타겟으로 분류
- 당뇨병 여부인 Outcome 속성이 타겟이므로 추출하여 target에 저장
- pop 함수를 사용했기 때문에 df에서 Outcome 속성이 삭제되어 df에는 특성만 남음

코드 14-5

```
01 target = df.pop('Outcome')
```

2. 딥러닝 분류

■ 당뇨병 발병 예측하기

■ 데이터 전처리하기

- 타깃은 0과 1의 숫자 레이블로 구성되는데, 의미는 다음과 같음

표 14-2 당뇨병 데이터 세트의 타깃

레이블	0	1
의미	당뇨병 없음	당뇨병 있음

2. 딥러닝 분류

■ 당뇨병 발병 예측하기

■ 데이터 전처리하기

- 사이킷런의 `train_test_split` 함수를 이용해서 특성인 `df`와 타깃인 `target`을 훈련 세트의 특성과 타깃, 테스트 세트의 특성과 타깃으로 나눔

코드 14-6

```
01 from sklearn.model_selection import train_test_split
02
03 X_train, X_test, y_train, y_test = train_test_split(df, target, test_size=0.2,
04                                                    random_state=0)
05 print(X_train.shape, y_train.shape)
06 print(X_test.shape, y_test.shape)
```

```
(313, 8) (313,)
(79, 8) (79,)
```

2. 딥러닝 분류

■ 당뇨병 발병 예측하기

■ 데이터 전처리하기

- X_train과 y_train은 훈련 세트이고 X_test와 y_test는 테스트 세트, X_train과
- X_test는 특성이고 y_train과 y_test는 타깃

X_train, X_test, y_train, y_test = train_test_split(df, target, ...)

테스트 세트의 특성 테스트 세트의 타깃 데이터 세트의 타깃

훈련 세트의 특성 훈련 세트의 타깃 데이터 세트의 특성

2. 딥러닝 분류

■ 당뇨병 발병 예측하기

■ 데이터 전처리하기

- 특성 값의 범위 차이가 크면 일부 특성에 크게 의존하게 되어 모델을 최적의 성능으로 만들지 못함
- 이런 이유로 특성의 범위를 유사하게 하는 데이터 스케일링을 진행
- 여러 종류의 데이터 스케일러 중 대표적으로 사용되는 것이 사이킷런의 StandardScaler 함수

| Syntax | StandardScaler() 함수

```
from sklearn.preprocessing import StandardScaler  
scaler = StandardScaler()
```

데이터 스케일러를 설정합니다.

2. 딥러닝 분류

■ 당뇨병 발병 예측하기

■ 데이터 전처리하기

- 훈련 세트 특성과 테스트 세트 특성의 스케일링을 진행
- fit_transform과 transform은 둘 다 스케일링하는 함수인데, fit_transform은 특성의 평균과 표준편차를 계산한 후에 스케일링

코드 14-7

```
01 from sklearn.preprocessing import StandardScaler
02
03 scaler = StandardScaler()
04 X_train = scaler.fit_transform(X_train)
05 X_test = scaler.transform(X_test)
```


2. 딥러닝 분류

■ 당뇨병 발병 예측하기

▪ 모델 생성하기

- 데이터 세트가 준비되었으므로 당뇨병 발병을 예측하는 딥러닝 모델을 만들면서 모델 생성 과정을 살펴보기

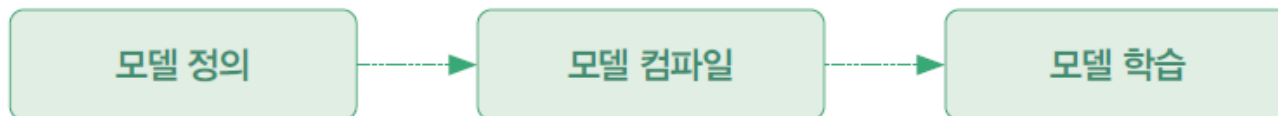


그림 14-8 딥러닝 모델 생성 절차

2. 딥러닝 분류

■ 당뇨병 발병 예측하기

▪ 모델 정의하기

- 딥러닝은 여러 층(layer) 이 쌓인 구조인데, 케라스에서는 일반적으로 Sequential 함수를 이용해서 순차적인 모델을 생성한 후에 층을 추가해서 딥러닝 모델을 정의

| Syntax | Sequential() 함수

```
from tensorflow.keras.models import Sequential  
model = Sequential()
```

순차적인 딥러닝 모델을 생성합니다.

2. 딥러닝 분류

■ 당뇨병 발병 예측하기

▪ 모델 정의하기

- 모델을 생성했으면 add 함수를 이용해서 층을 추가
- 신경망에서 층을 구성할 때 사용하는 대표적인 함수는 Dense
- Dense 함수는 입력과 출력을 연결하는데, 입력층과 출력층의 모든 노드가 서로 연결된 완전 연결층(fully connected layer) 을 생성
- 함수의 매개변수로 해당층의 노드 수를 설정, activation에 활성화 함수를 설정
- input_dim을 이용해서 입력층의 입력 수를 설정

| Syntax | Dense() 함수

```
from tensorflow.keras.layers import Dense  
Dense(해당 층의 노드 수, activation=활성화 함수, input_dim=입력 수)
```

신경망의 층으로 입력과 출력을 연결합니다.

2. 딥러닝 분류

■ 당뇨병 발병 예측하기

■ 모델 정의하기

- 활성화 함수는 입력한 값을 다음 층으로 넘길 때 각 값을 어떻게 처리할지를 결정하는 함수
- 이진 분류 문제에서는 활성화 함수로 sigmoid를 사용하고 다중 분류 문제에서는 softmax를 사용
- 은닉층에서는 주로 relu를 사용

표 14-3 활성화 함수

활성화 함수	설명
sigmoid	이진 분류 문제에서 사용한다.
softmax	다중 분류 문제에서 사용한다.
relu	은닉층에서 주로 사용한다.

2. 딥러닝 분류

■ 당뇨병 발병 예측하기

▪ 모델 정의하기

- 간단한 예를 통해서 입력 수와 해당 층의 노드 수의 관계를 알아보기
- 마지막 층을 출력층이라 하고 나머지 층을 은닉층이라 함
- 첫 번째 층은 은닉층과 입력층 역할을 겸함

```
model = Sequential() ..... 순차 모델 생성  
model.add(Dense(3, ..., input_dim=2)) ..... 입력층, 은닉층  
model.add(Dense(1, ...)) ..... 출력층
```

그림 14-9 딥러닝 모델의 층 구조

2. 딥러닝 분류

■ 당뇨병 발병 예측하기

■ 모델 정의하기

- 첫 번째 층인 Dense(3, ..., input_dim=2)에서 input_dim 값인 2가 입력 수이고 3이 해당 층의 노드 수
- 두 번째 층인 Dense(1, ...)은 마지막 층이므로 출력층이고, 1이 해당 층의 노드 수
- 입력 값이 노드로 그대로 전달되는 것은 아니고 층에서 설정한 활성화 함수의 계산 방법에 따라 계산한 값을 전달

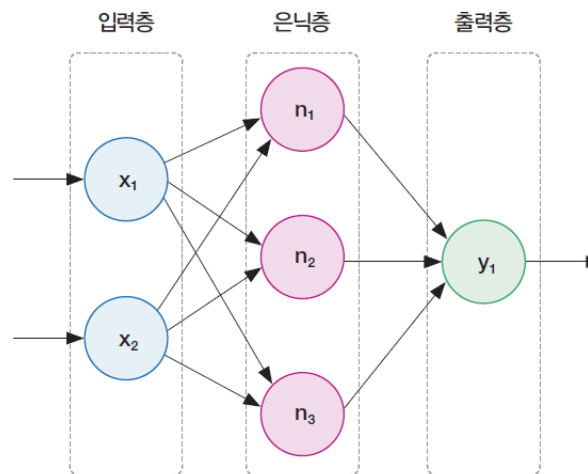


그림 14-10 딥러닝 모델의 구조

2. 딥러닝 분류

■ 당뇨병 발병 예측하기

▪ 모델 정의하기

- 당뇨병 발병을 예측하는 딥러닝 모델을 만들기
- model이라는 모델을 생성한 후에 add 함수를 이용해서 3개의 층을 추가

코드 14-8

```
01 from tensorflow.keras.models import Sequential
02 from tensorflow.keras.layers import Dense
03
04 model = Sequential()
05 model.add(Dense(12, activation='relu', input_dim=8))
06 model.add(Dense(8, activation='relu'))
07 model.add(Dense(1, activation='sigmoid'))
```

2. 딥러닝 분류

■ 당뇨병 발병 예측하기

■ 모델 정의하기

- 마지막 층이 출력층이고 나머지 두 개의 층이 은닉층
- 은닉층과 입력층 역할을 겸하는 첫 번째 층에서는 input_dim을 이용해서 입력 수를 설정하는데 샘플의 특성 개수가 8이므로 8로 설정
- 첫 번째 층의 노드 수인 12와 두 번째 층의 노드 수인 8은 임의로 설정했으니 값을 변경하면서 최적의 값을 찾아내야 함
- 이 모델은 최종적으로 당뇨병인지 아닌지를 나타내는 하나의 값을 출력하므로 출력층의 노드 수는 1로 설정

```
model.add(Dense(12, activation='relu', input_dim=8))
model.add(Dense(8, activation='relu'))
model.add(Dense(1, activation='sigmoid'))
```

입력 수 설정

입력층, 은닉층

은닉층

출력층

노드 수 설정

그림 14-11 딥러닝 모델의 정의

2. 딥러닝 분류

■ 당뇨병 발병 예측하기

▪ 모델 정의하기

- 첫 번째 층과 두 번째 층의 활성화 함수는 가장 많이 사용하는 relu로 설정하고, 당뇨병 여부를 판별하는 이진 분류 문제이므로 출력층에서는 sigmoid를 활성화 함수로 설정
- 샘플의 특성인 임신 횟수, 포도당 수치, 혈압, 피부 두께 등 8개 값이 입력층을 통해 입력되고, 출력층의 출력 노드를 통해 당뇨병 여부 예측 값 1개가 출력됨

2. 딥러닝 분류

■ 당뇨병 발병 예측하기

■ 모델 정의하기

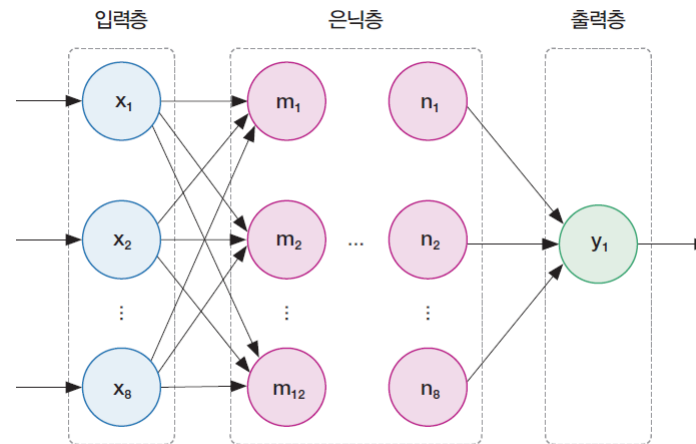


그림 14-12 model의 층 구조

```
model = Sequential([
    Dense(12, activation='relu', input_dim=8),
    Dense(8, activation='relu'),
    Dense(1, activation='sigmoid')
])
```

2. 딥러닝 분류

■ 당뇨병 발병 예측하기

▪ 모델 컴파일하기

- 모델 컴파일은 모델의 학습 환경을 설정하는 과정
- 최적화함수, 손실함수, 측정지표를 적절하게 설정하면 모델의 성능을 향상시킬 수 있음
- 케라스에서는 compile 함수를 이용해서 모델을 컴파일

| Syntax | compile() 함수

```
model.compile(optimizer=최적화함수, loss=손실함수, metrics=측정지표)
```

모델의 학습 환경을 설정합니다.

2. 딥러닝 분류

■ 당뇨병 발병 예측하기

■ 모델 컴파일하기

- 최적화 함수(optimizer) 는 입력 데이터를 기반으로 모델을 최적화하는 방법을 결정하는 함수
- 최적화 함수는 손실 함수의 값을 최소화하는 방향으로 모델을 업데이트
- 대표적인 최적화 함수는 [표 14-4]와 같고 이 중에서 'Adam'을 자주 사용

표 14-4 최적화 함수

최적화 함수	설명
SGD(Stochastic Gradient Descent)	가장 기본적인 최적화 방법이다. 미분 가능한 함수에서 기울기를 계산하여 모델을 업데이트한다.
Momentum	SGD에 가속도를 추가한 방법이다. 모델을 업데이트할 때 이전의 업데이트 방향을 고려하여 더 빠르게 학습한다.
RMSprop(Root Mean Square Propagation)	SGD의 변형이다. 기울기 제곱근의 평균을 사용하여 모델을 업데이트한다.
Adam(Adaptive Moment Estimation)	Momentum과 RMSprop의 장점을 접목한다. 기울기 값과 기울기 제곱의 지수이동평균을 활용하여 모델을 업데이트한다.

2. 딥러닝 분류

■ 당뇨병 발병 예측하기

■ 모델 컴파일하기

- 손실 함수는 모델의 예측 값과 실제 값의 차이를 측정하는 함수
- 손실 함수를 사용하여 모델의 성능을 평가하며 모델을 최적화

표 14-5 손실 함수

손실 함수	설명
mean_squared_error	예측 값과 실제 값 사이의 평균 제곱 오차를 계산하며, 회귀 문제에서 사용한다.
mean_absolute_error	예측 값과 실제 값 사이의 평균 절대 오차를 계산하며, 회귀 문제에서 사용한다.
binary_crossentropy	예측 값과 실제 값 사이의 이진 크로스 엔트로피를 계산하며, 이진 분류 문제에서 사용한다.
categorical_crossentropy	예측 값과 실제 값 사이의 카테고리 크로스 엔트로피를 계산하며, 다중 분류 문제 중 타깃이 원-핫 인코딩인 경우에 사용한다.
sparse_categorical_crossentropy	예측 값과 실제 값 사이의 카테고리 크로스 엔트로피를 계산하며, 다중 분류 문제 중 타깃이 하나의 정수인 경우에 사용한다.

2. 딥러닝 분류

■ 당뇨병 발병 예측하기

▪ 모델 컴파일하기

- 측정 지표는 모델의 성능을 측정하는 지표
- 측정 지표를 사용하여 모델의 성능을 평가하고 모델을 개선할 방향을 결정

표 14-6 측정 지표

측정 지표	설명
accuracy	실제 값과 예측 값이 일치하는 비율이며, 분류 문제에서 사용한다.
precision	실제 값이 양성인 데이터 중에서 예측도 양성으로 맞춘 비율이며, 분류 문제에서 사용한다.
mse(mean squared error)	실제값과 예측값의 차를 제곱한 값의 평균이며, 회귀 문제에서 사용한다.
mae(mean absolute error)	실제값과 예측값의 차이에 대한 평균이며, 회귀 문제에서 사용한다.

2. 딥러닝 분류

■ 당뇨병 발병 예측하기

▪ 모델 컴파일하기

- [코드 14-9]는 앞에서 정의한 model에 최적화 함수, 손실 함수, 측정 지표를 설정하여 컴파일
- 모델은 Adam 최적화 방법을 사용하여 binary_crossentropy를 최소화하는 방향으로 학습하게 되며 이때 모델 성능은 accuracy로 측정

코드 14-9

```
01 model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
```

2. 딥러닝 분류

■ 당뇨병 발병 예측하기

▪ 모델 학습시키기

- 모델을 정의하고 컴파일하고 나면 훈련 세트의 특성과 타깃을 이용해서 학습시켜야 모델이 완성됨
- 모델 학습에는 fit 함수를 이용
- 모델의 예측 값과 실제 값(타깃)의 오차를 줄이면서 최적의 모델을 찾게 됨

| Syntax | fit() 함수

```
모델.fit(특성, 타깃, epochs=학습횟수, batch_size=배치 사이즈,  
        validation_data=검증데이터)
```

‘모델’을 학습시킨다.

2. 딥러닝 분류

■ 당뇨병 발병 예측하기

▪ 모델 학습시키기

- 학습 횟수는 epochs를 이용해서 설정하는데, 전체 데이터 세트를 몇 번 반복해서 학습할지 정수값을 입력
- 배치 사이즈는 batch_size를 이용해서 설정하는데, 모델을 갱신하기 위해 처리하는 데이터의 수를 의미
- 검증 데이터는 validation_data를 이용해서 설정하는데, 모델의 성능을 검증하기 위한 데이터 세트
- 매 반복(epoch)마다 훈련 세트와 검증 데이터서의 손실값과 정확도를 확인할 수 있음

2. 딥러닝 분류

■ 당뇨병 발병 예측하기

■ 모델 학습시키기

- [코드 14-10]은 앞에서 정의하고 컴파일한 model을 학습시킴
- 모델의 예측 값과 타겟의 오차를 줄이면서 최적의 모델을 찾음
- 반복할 때마다 실행 결과를 history에 저장

코드 14-10

```
01 history = model.fit(X_train, y_train, epochs=30, batch_size=5,\n02                     validation_data=(X_test, y_test))
```

```
history = model.fit(X_train, y_train, epochs=30, batch_size=5, validation_data=(X_test, y_test))
```

훈련 세트

학습 횟수 설정

배치 사이즈 설정

검증 데이터 설정

그림 14-13 모델 학습 코드의 구조

2. 딥러닝 분류

■ 당뇨병 발병 예측하기

▪ 모델 학습시키기

```
Epoch 1/30
63/63 [=====] - 1s 7ms/step - loss: 0.5901 - accuracy: 0.6677 -
val_loss: 0.5615 - val_accuracy: 0.6835
Epoch 2/30
63/63 [=====] - 0s 5ms/step - loss: 0.5371 - accuracy: 0.6805 -
val_loss: 0.5196 - val_accuracy: 0.7089
Epoch 3/30
63/63 [=====] - 0s 5ms/step - loss: 0.5096 - accuracy: 0.7157 -
val_loss: 0.4982 - val_accuracy: 0.7468

:

Epoch 29/30
63/63 [=====] - 0s 7ms/step - loss: 0.3973 - accuracy: 0.8307 -
val_loss: 0.4549 - val_accuracy: 0.7722
Epoch 30/30
63/63 [=====] - 0s 6ms/step - loss: 0.3950 - accuracy: 0.8275 -
val_loss: 0.4551 - val_accuracy: 0.7975
```

2. 딥러닝 분류

■ 당뇨병 발병 예측하기

▪ 모델 학습시키기

- 실행 결과에서 loss는 훈련 세트의 손실값, accuracy는 훈련 세트의 정확도, val_loss는 검증 데이터의 손실값, val_accuracy는 검증 데이터의 정확도를 의미
- 층 수, 각 층의 출력 노드 수, 학습 횟수, 배치 사이즈를 다양하게 변경하면서 훈련 세트와 검증데이터 모두에서 좋은 성능을 보이는 최적의 값을 찾아야 함

2. 딥러닝 분류

■ 당뇨병 발병 예측하기

▪ 모델 평가하기

- 케라스에서 모델의 성능을 평가할 때 evaluate 함수를 이용
- 이 함수는 손실값(loss), 정확도(accuracy) 등을 계산

| Syntax | evaluate() 함수

```
모델.evaluate(특성, 타겟)
```

‘모델’의 성능을 평가한다.

2. 딥러닝 분류

■ 당뇨병 발병 예측하기

▪ 모델 평가하기

- 이 모델의 훈련 세트와 테스트 세트의 성능을 평가하는 코드는 [코드 14-11]과 같음

코드 14-11

```
01 print(model.evaluate(X_train, y_train))  
02 print(model.evaluate(X_test, y_test))
```

```
10/10 [=====] - 0s 3ms/step - loss: 0.3898 - accuracy: 0.8275  
[0.3897651135921478, 0.8274760246276855]  
3/3 [=====] - 0s 4ms/step - loss: 0.4551 - accuracy: 0.7975  
[0.4550565779209137, 0.797468364238739]
```

2. 딥러닝 분류

■ 당뇨병 발병 예측하기

▪ 모델 평가하기

- [코드 14-12]는 매 반복에서 얻은 훈련 세트와 검증 데이터의 손실값 및 정확도를 그래프로 나타냄
- 이번에는 `history.history`에 손실값과 정확도를 저장
- 훈련 세트의 손실값은 `history.history['loss']`, 정확도는 `history.history['accuracy']`로, 검증 데이터에 대한 손실값은 `history.history['val_loss']`, 정확도는 `history.history['val_accuracy']`로 확인할 수 있음

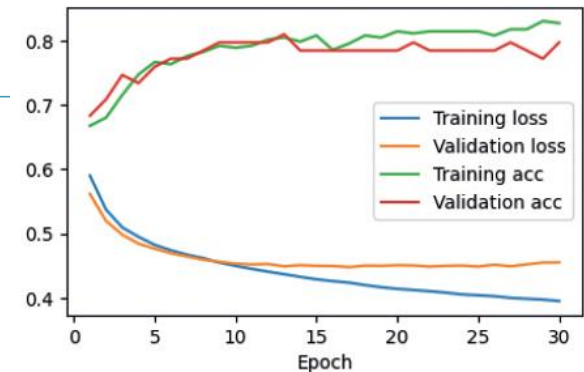
2. 딥러닝 분류

■ 당뇨병 발병 예측하기

▪ 모델 평가하기

코드 14-12

```
01 import matplotlib.pyplot as plt
02
03 epochs = np.arange(1, len(history.history['loss']) + 1)
04
05 plt.figure(figsize=(5,3))
06 plt.plot(epochs, history.history['loss'], label='Training loss')
07 plt.plot(epochs, history.history['val_loss'], label='Validation loss')
08 plt.plot(epochs, history.history['accuracy'], label='Training acc')
09 plt.plot(epochs, history.history['val_accuracy'], label='Validation acc')
10 plt.xlabel('Epoch')
11 plt.legend()
12 plt.show()
```



2. 딥러닝 분류

LAB 붓꽃 품종 분류하기

- 딥러닝을 이용해서 붓꽃 품종을 분류하는 프로그램을 만들어 보기
- 사이킷런 라이브러리에서 제공하는 붓꽃 데이터 세트를 활용
- 꽃받침 길이(sepal length), 꽃받침 너비(sepal width), 꽃잎길이(petal length), 꽃잎 너비(petal width), 품종으로 이루어짐

표 14-7 붓꽃 데이터 세트

(단위: cm)

꽃받침 길이	꽃받침 너비	꽃잎 길이	꽃잎 너비	품종
5.1	3.5	1.4	0.2	세토사
4.9	3	1.4	0.2	세토사

2. 딥러닝 분류

LAB 붓꽃 품종 분류하기

- 붓꽃을 3가지 중 한 품종으로 분류하는 다중 분류 문제이므로 딥러닝 모델 출력층의 활성화 함수는 softmax로 설정하여 정의해야 함

코드 14-13

```
01 from sklearn import datasets
02 from sklearn.model_selection import train_test_split
03 from tensorflow.keras.models import Sequential
04 from tensorflow.keras.layers import Dense
```

2. 딥러닝 분류

LAB 붓꽃 품종 분류하기

```
05 import numpy as np
06 import matplotlib.pyplot as plt
07
08 # 데이터 수집하기
09 iris = datasets.load_iris()
10 print(iris['data'][:5])
11 print(iris['target'][:5])
12
13 # 데이터 전처리하기
14 X_train, X_test, y_train, y_test = train_test_split(iris['data'], iris['target'],\
15                                                    random_state=0)
16 print(X_train.shape, y_train.shape)
17 print(X_test.shape, y_test.shape)
18
19 # 모델 생성하기
20 model = Sequential()
21 model.add(Dense(64, activation='relu', input_dim=4))
22 model.add(Dense(64, activation='relu'))
23 model.add(Dense(3, activation='softmax'))
```

2. 딥러닝 분류

LAB 붓꽃 품종 분류하기

```
24
25 model.compile(optimizer='adam', loss='sparse_categorical_crossentropy',\
26               metrics=['accuracy'])
27 history = model.fit(X_train, y_train, epochs=50, validation_data=(X_test, y_test))
28
29 # 모델 평가하기
30 print(model.evaluate(X_train, y_train))
31 print(model.evaluate(X_test, y_test))
32
33 epochs = np.arange(1, len(history.history['loss']) + 1)
34 plt.figure(figsize=(7,4))
35 plt.plot(epochs, history.history['loss'], label='Training loss')
36 plt.plot(epochs, history.history['val_loss'], label='Validation loss')
37 plt.plot(epochs, history.history['accuracy'], label='Training acc')
38 plt.plot(epochs, history.history['val_accuracy'], label='Validation acc')
39 plt.xlabel('Epoch')
40 plt.legend()
41 plt.show()
```

2. 딥러닝 분류

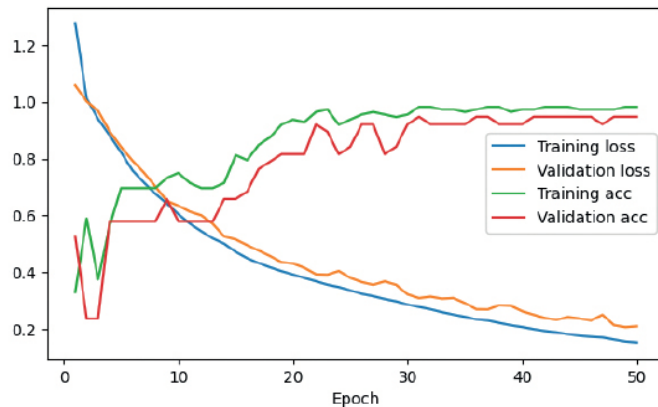
LAB 붓꽃 품종 분류하기

```
[[5.1 3.5 1.4 0.2]
 [4.9 3. 1.4 0.2]
 [4.7 3.2 1.3 0.2]
 [4.6 3.1 1.5 0.2]
 [5. 3.6 1.4 0.2]]
[0 0 0 0]
(112, 4) (112,)
(38, 4) (38,)
Epoch 1/50
4/4 [=====] - 1s 158ms/step - loss: 1.2768 - accuracy: 0.3304 -
val_loss: 1.0578 - val_accuracy: 0.5263
Epoch 2/50
4/4 [=====] - 0s 14ms/step - loss: 1.0113 - accuracy: 0.5893 - val_
loss: 1.0018 - val_accuracy: 0.2368
Epoch 3/50
4/4 [=====] - 0s 19ms/step - loss: 0.9392 - accuracy: 0.3750 - val_
loss: 0.9680 - val_accuracy: 0.2368
:
Epoch 49/50
4/4 [=====] - 0s 20ms/step - loss: 0.1564 - accuracy: 0.9821 - val_
loss: 0.2071 - val_accuracy: 0.9474
Epoch 50/50
4/4 [=====] - 0s 20ms/step - loss: 0.1532 - accuracy: 0.9821 - val_
loss: 0.2100 - val_accuracy: 0.9474
```

2. 딥러닝 분류

LAB 붓꽃 품종 분류하기

4/4 ===== - 0s 4ms/step - loss: 0.1486 - accuracy: 0.9821
[0.14863526821136475, 0.9821428656578064]
2/2 ===== - 0s 7ms/step - loss: 0.2100 - accuracy: 0.9474
[0.21002161502838135, 0.9473684430122375]



딥러닝 회귀

3. 딥러닝 회귀

■ 보스턴 주택 가격 예측하기

- 미국 보스턴의 주택 가격을 예측하는 딥러닝 모델 사례를 통해 딥러닝 회귀를 살펴보기
- 보스턴 주택 가격 데이터 세트는 1970년 중반 보스턴 지역의 1인당 범죄율, 주택당 평균 방의 개수, 고속도로 접근성, 재산세율, 학생-교사 비율, 평균 주택 가격 등을 포함하는 데이터
- 404개의 훈련 세트와 102개의 테스트 세트로 이미 나뉘어져 있음
- 특성은 총 13개이고 타겟은 평균 주택 가격

3. 딥러닝 회귀

■ 보스턴 주택 가격 예측하기

■ 데이터 수집하기

- 케라스 라이브러리에서 제공하는 보스턴 주택 데이터 세트를 불러오기

코드 14-14

```
01 from tensorflow.keras.datasets import boston_housing
02
03 (X_train, y_train), (X_test, y_test) = boston_housing.load_data()
```

3. 딥러닝 회귀

■ 보스턴 주택 가격 예측하기

■ 데이터 수집하기

- 훈련 세트의 특성인 X_{train} , 훈련 세트의 타깃인 y_{train} , 테스트 세트의 특성인 X_{test} , 테스트 세트의 타깃인 y_{test} 의 크기를 확인
- 404개의 훈련 세트와 102개의 테스트 세트로 이루어지며, 각 데이터는 13개의 특성과 1개의 타깃으로 구성됨

코드 14-15

```
01 print(X_train.shape, y_train.shape)
02 print(X_test.shape, y_test.shape)
```

```
(404, 13) (404,)
(102, 13) (102,)
```

3. 딥러닝 회귀

■ 보스턴 주택 가격 예측하기

■ 데이터 수집하기

- 훈련 세트 첫 번째 샘플의 특성인 `X_train[0]`과 첫 번째 샘플의 타깃인 `y_train[0]`을 확인

코드 14-16

```
01 print(X_train[0])  
02 print(y_train[0])
```

```
[-0.27224633 -0.48361547 -0.43576161 -0.25683275 -0.1652266 -0.1764426  
 0.81306188 0.1166983 -0.62624905 -0.59517003 1.14850044 0.44807713  
 0.8252202 ]  
15.2
```

3. 딥러닝 회귀

■ 보스턴 주택 가격 예측하기

■ 데이터 전처리

- 훈련 세트 특성과 테스트 세트 특성의 범위를 비슷하게 하는 데이터 스케일링을 진행
- fit_transform과 transform은 둘 다 데이터를 스케일링하는 함수인데 fit_transform 함수는 평균과 표준편차를 계산한 후에 스케일링을 진행

코드 14-17

```
01 from sklearn.preprocessing import StandardScaler
02
03 scaler = StandardScaler()
04 X_train = scaler.fit_transform(X_train)
05 X_test = scaler.transform(X_test)
```

3. 딥러닝 회귀

■ 보스턴 주택 가격 예측하기

▪ 모델 생성하기

- 주택 가격을 예측하는 딥러닝 모델을 만들어보기

① **모델 정의하기:** Sequential 함수를 이용해서 순차적인 모델인 model을 생성한 후에 add 함수를 이용해서 3개의 Dense 층을 추가

코드 14-18

```
01 from tensorflow.keras.models import Sequential
02 from tensorflow.keras.layers import Dense
03
04 model = Sequential()
05 model.add(Dense(64, activation='relu', input_dim=13))
06 model.add(Dense(64, activation='relu'))
07 model.add(Dense(1))
```

3. 딥러닝 회귀

■ 보스턴 주택 가격 예측하기

■ 모델 생성하기

① 모델 정의하기

- 마지막 층이 출력층이고 나머지 두 층이 은닉층

```
model.add(Dense(64, activation='relu', input_dim=13))
model.add(Dense(64, activation='relu'))
model.add(Dense(1))
```

노드 수 설정 입력 수 설정

입력층, 은닉층

은닉층

출력층

그림 14-14 딥러닝 모델의 정의

3. 딥러닝 회귀

■ 보스턴 주택 가격 예측하기

■ 모델 생성하기

① 모델 정의하기

- 은닉층과 입력층을 겸하는 첫 번째 층에서 input_dim을 이용해서 입력 수를 설정.
- 샘플의 특성 개수가 13이므로 입력 수를 13으로 설정.
- 첫 번째 층과 두 번째 층의 활성화 함수는 relu로 설정했으며, 노드 수는 임의로 64를 설정

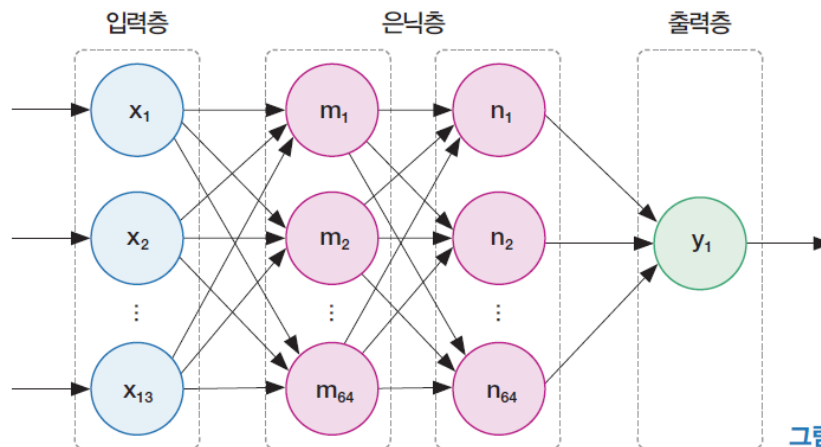


그림 14-15 model의 층 구조

3. 딥러닝 회귀

■ 보스턴 주택 가격 예측하기

▪ 모델 생성하기

② 모델 컴파일하기

- 최적화 함수를 adam으로 설정하고, 회귀 문제이므로 손실 함수를 mse(mean_squared_error)로 설정
- 측정 지표를 mae(mean absolute error)로 하여 모델의 학습 환경을 설정

코드 14-19

```
01 model.compile(optimizer='adam', loss='mse', metrics=['mae'])
```


3. 딥러닝 회귀

■ 보스턴 주택 가격 예측하기

▪ 모델 생성하기

③ 모델 학습시키기

- model을 특성 X_train과 타겟 y_train을 이용해서 학습시키기

코드 14-20

```
01 history = model.fit(X_train, y_train, epochs=50, batch_size=16, validation_split=0.25)
```

```
Epoch 1/50
19/19 [=====] - 1s 15ms/step - loss: 542.8005 - mae: 21.6128 -
val_loss: 644.5643 - val_mae: 23.4291
Epoch 2/50
19/19 [=====] - 0s 4ms/step - loss: 481.8712 - mae: 20.2466 - val_
loss: 567.0059 - val_mae: 21.8546
Epoch 3/50
19/19 [=====] - 0s 5ms/step - loss: 401.0137 - mae: 18.2705 - val_
loss: 447.2325 - val_mae: 19.1445
```

3. 딥러닝 회귀

■ 보스턴 주택 가격 예측하기

▪ 모델 생성하기

③ 모델 학습시키기

- model을 특성 X_{train} 과 타겟 y_{train} 을 이용해서 학습시키기

```
⋮
```

```
Epoch 49/50
```

```
19/19 [=====] - 0s 4ms/step - loss: 8.0243 - mae: 2.0007 - val_
```

```
loss: 11.5740 - val_mae: 2.5151
```

```
Epoch 50/50
```

```
19/19 [=====] - 0s 4ms/step - loss: 7.9219 - mae: 2.0009 - val_
```

```
loss: 11.5058 - val_mae: 2.5038
```

3. 딥러닝 회귀

■ 보스턴 주택 가격 예측하기

▪ 모델 생성하기

④ 모델 평가하기

- 이 모델의 훈련 세트에서의 성능과 테스트 세트에서의 성능을 평가하는 코드는 다음과 같음

코드 14-21

```
01 print(model.evaluate(X_train, y_train))  
02 print(model.evaluate(X_test, y_test))
```

```
13/13 [=====] - 0s 3ms/step - loss: 8.6536 - mae: 2.0961  
[8.653585433959961, 2.096081018447876]  
4/4 [=====] - 0s 3ms/step - loss: 21.5485 - mae: 2.9226  
[21.548500061035156, 2.922603130340576]
```

3. 딥러닝 회귀

■ 보스턴 주택 가격 예측하기

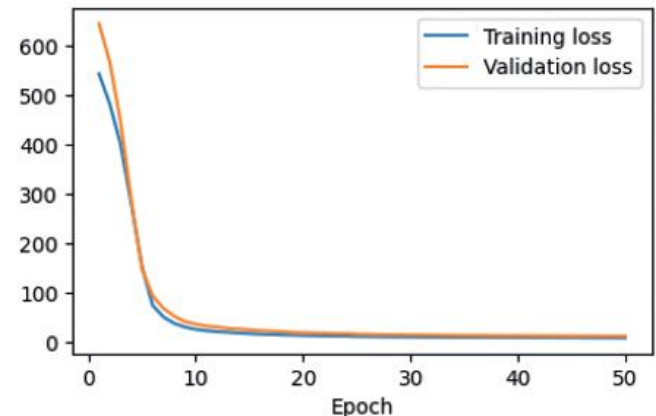
■ 모델 생성하기

④ 모델 평가하기

- 매 반복에서 훈련 세트와 검증 데이터의 손실값을 그래프로 나타냄
- 훈련 세트의 손실값은 loss로, 검증 데이터에 대한 손실값은 val_loss로 확인할 수 있음

코드 14-22

```
01 import numpy as np
02 import matplotlib.pyplot as plt
03
04 epochs = np.arange(1, len(history.history['loss']) + 1)
05
06 plt.figure(figsize=(5,3))
07 plt.plot(epochs, history.history['loss'], label='Training loss')
08 plt.plot(epochs, history.history['val_loss'], label='Validation loss',
09 plt.xlabel('Epoch')
10 plt.legend()
11 plt.show()
```



3. 딥러닝 회귀

■ 보스턴 주택 가격 예측하기

▪ 모델 생성하기

- predict 함수를 이용해서 주택의 예측 가격을 구해 보기
- [코드 14-23]은 테스트 세트의 특성을 이용해서 주택 예측 가격을 구하고 y_pred에 저장
- 테스트 세트의 특성으로 구한 모델의 주택 예측 가격과 테스트 세트의 실제 주택 가격을 그래프로 비교

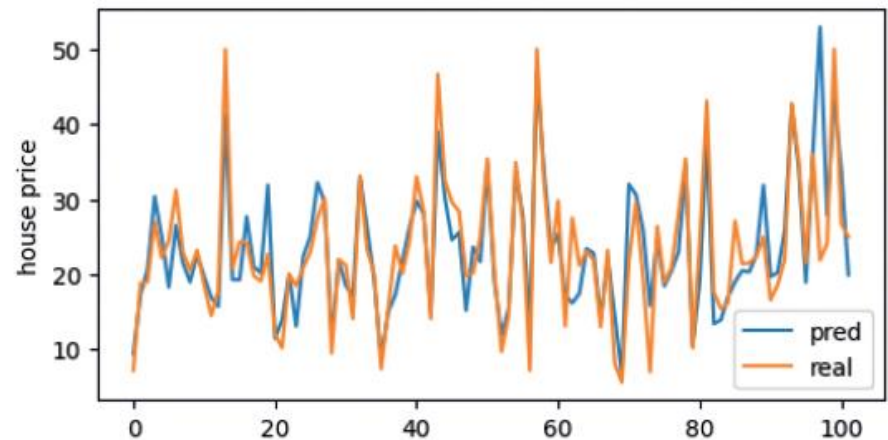
3. 딥러닝 회귀

■ 보스턴 주택 가격 예측하기

▪ 모델 생성하기

코드 14-23

```
01 y_pred = model.predict(X_test)
02 ind = np.arange(len(y_pred))
03
04 plt.figure(figsize=(6,3))
05 plt.plot(ind, y_pred, label='pred')
06 plt.plot(ind, y_test, label='real')
07 plt.ylabel('house price')
08 plt.legend()
09 plt.show()
```



3. 머신러닝 회귀

LAB 자동차 연비 예측하기

- 딥러닝을 이용해서 자동차 연비를 예측하는 프로그램을 만들어보기
- Seaborn 라이브러리에서 제공하는 Auto MPG 데이터 세트를 활용
- 중 실린더 수, 배기량, 마력 등의 정보로 연비를 예측할 것이므로 연비인 mpg가 타깃

표 14-8 Auto MPG 데이터 세트의 속성

속성 이름	설명	속성 이름	설명	속성 이름	설명
mpg	연비	horsepower	마력	model year	출시년도
cylinders	실린더 수	weight	무게	origin	제조국
displacement	배기량	acceleration	가속도	name	모델명

3. 머신러닝 회귀

LAB 자동차 연비 예측하기

- 회귀 문제이므로 모델의 출력층에서는 활성화 함수를 지정하지 않고 출력 노드 수를 1로 설정

코드 14-24

```
01 import seaborn as sns
02 from sklearn.model_selection import train_test_split
03 from sklearn.preprocessing import StandardScaler
04 from tensorflow.keras.models import Sequential
05 from tensorflow.keras.layers import Dense
06 import numpy as np
07 import matplotlib.pyplot as plt
08
09 # 데이터 수집하기
10 mpg = sns.load_dataset('mpg')
11 display(mpg.head())
12
13 # 데이터 전처리하기
14 mpg.isnull().sum()
15 mpg = mpg.dropna()
```


3. 머신러닝 회귀

LAB 자동차 연비 예측하기

```
16 target = mpg.pop('mpg')
17 data = mpg.drop(columns=['origin', 'name'])
18 display(data.head())
19
20 X_train, X_test, y_train, y_test = train_test_split(data, target, random_
    state=0)
21 print(X_train.shape, y_train.shape)
22 print(X_test.shape, y_test.shape)
23
24 scaler = StandardScaler()
25 X_train = scaler.fit_transform(X_train)
26 X_test = scaler.transform(X_test)
27
28 # 모델 생성하기
29 model = Sequential()
30 model.add(Dense(64, activation='relu', input_dim=6))
31 model.add(Dense(64, activation='relu'))
32 model.add(Dense(1))
33
```

3. 머신러닝 회귀

LAB 자동차 연비 예측하기

```
34 model.compile(optimizer='adam', loss='mse', metrics=['mae'])
35 history = model.fit(X_train, y_train, epochs=100, validation_data=(X_test, y_
    test))
36
37 # 모델 평가하기
38 print(model.evaluate(X_train, y_train))
39 print(model.evaluate(X_test, y_test))
40
41 epochs = np.arange(1, len(history.history['loss']) + 1)
42 plt.figure(figsize=(5,3))
43 plt.plot(epochs, history.history['loss'], label='Training loss')
44 plt.plot(epochs, history.history['val_loss'], label='Validation loss')
45 plt.xlabel('Epoch')
46 plt.legend()
47 plt.show()
48
49 y_pred = model.predict(X_test)
50 ind = np.arange(len(y_pred))
51 plt.figure(figsize=(6,3))
52 plt.plot(ind, y_pred, label='pred')
```

3. 머신러닝 회귀

LAB 자동차 연비 예측하기

```
53 plt.plot(ind, y_test, label='real')
54 plt.ylabel('MPG')
55 plt.legend()
56 plt.show()
```

	mpg	cylinders	displacement	horsepower	weight	acceleration	model_year	origin	name
0	18.0	8	307.0	130.0	3504	12.0	70	usa	chevrolet chevelle malibu
1	15.0	8	350.0	165.0	3693	11.5	70	usa	buick skylark 320
2	18.0	8	318.0	150.0	3436	11.0	70	usa	plymouth satellite
3	16.0	8	304.0	150.0	3433	12.0	70	usa	amc rebel sst
4	17.0	8	302.0	140.0	3449	10.5	70	usa	ford torino

```
mpg 0
cylinders 0
displacement 0
horsepower 6
weight 0
acceleration 0
model_year 0
origin 0
name 0
dtype: int64
```

3. 머신러닝 회귀

LAB 자동차 연비 예측하기

	cylinders	displacement	horsepower	weight	acceleration	model_year
0	8	307.0	130.0	3504	12.0	70
1	8	350.0	165.0	3693	11.5	70
2	8	318.0	150.0	3436	11.0	70
3	8	304.0	150.0	3433	12.0	70
4	8	302.0	140.0	3449	10.5	70

(294, 6) (294,)

(98, 6) (98,)

Epoch 1/100

10/10 [=====] - 1s 28ms/step - loss: 594.0882 - mae: 22.9635 -
val_loss: 591.3113 - val_mae: 22.9068

Epoch 2/100

10/10 [=====] - 0s 6ms/step - loss: 562.3729 - mae: 22.2221 - val_
loss: 557.2937 - val_mae: 22.0733

Epoch 3/100

10/10 [=====] - 0s 7ms/step - loss: 525.1105 - mae: 21.2974 - val_
loss: 514.9834 - val_mae: 21.0025

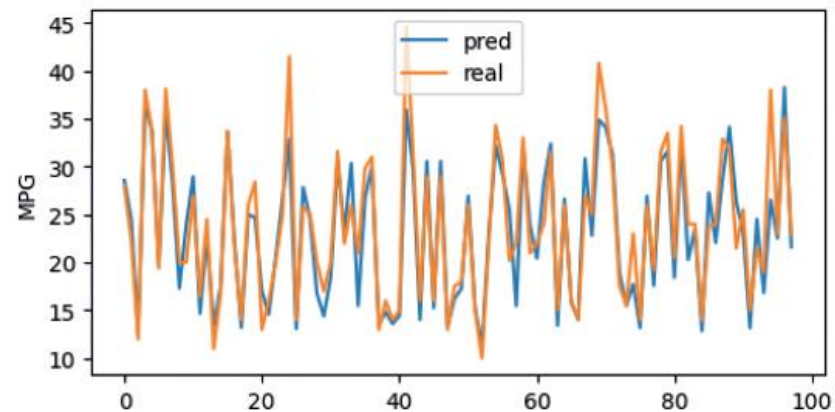
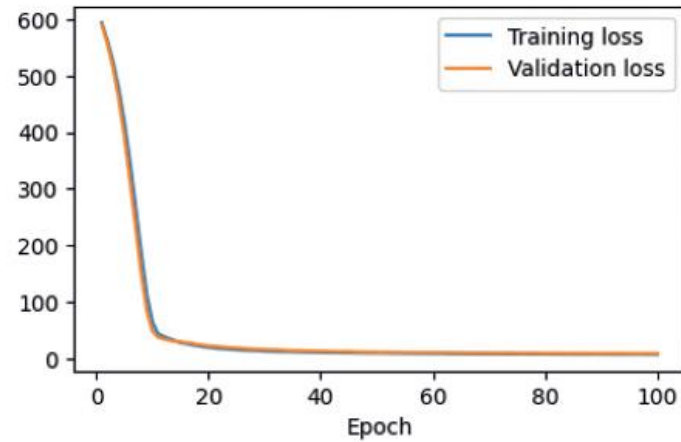
3. 머신러닝 회귀

LAB 자동차 연비 예측하기

```
⋮  
Epoch 99/100  
10/10 [=====] - 0s 7ms/step - loss: 7.1877 - mae: 1.9217 - val_  
loss: 8.5483 - val_mae: 2.1451  
Epoch 100/100  
10/10 [=====] - 0s 5ms/step - loss: 7.1594 - mae: 1.9101 - val_  
loss: 8.4943 - val_mae: 2.1339  
  
10/10 [=====] - 0s 3ms/step - loss: 7.0848 - mae: 1.9017  
[7.084806442260742, 1.9017242193222046]  
4/4 [=====] - 0s 5ms/step - loss: 8.4943 - mae: 2.1339  
[8.494274139404297, 2.1338987350463867]
```

3. 머신러닝 회귀

LAB 자동차 연비 예측하기



[플러스 예제] 당뇨병 진행도 예측

4. [플러스 예제] 당뇨병 진행도 예측

- 딥러닝을 이용해 당뇨병 진행도를 예측하는 프로그램을 만들어보기
- 당뇨병 데이터 세트는 10가지 특성으로 이루어져 있고 타겟은 1년 뒤에 측정한 당뇨병의 진행률

■ 데이터 수집하기

- 사이킷런 라이브러리에서 제공하는 당뇨병 데이터 세트를 불러와서 diabetes에 저장

```
01 from sklearn import datasets
02 diabetes = datasets.load_diabetes()
```


4. [플러스 예제] 당뇨병 진행도 예측

■ 데이터 수집하기

- diabetes 객체가 어떤 키로 이루어져 있는지 확인
- data는 442명의 피마 인디언의 나이, 성별, 체질량지수 등으로 당뇨병 진행률을 예측하기 위해 활용하는 특성에 해당하고, target은 data 각 행의 피마 인디언을 1년 뒤에 검사해서 얻은 당뇨병의 진행률을 의미

```
01 print('diabetes 키:', diabetes.keys())
```

```
diabetes 키: dict_keys(['data', 'target', 'frame', 'DESCR', 'feature_names', 'data_
filename', 'target_filename', 'data_module'])
```

4. [플러스 예제] 당뇨병 진행도 예측

■ 데이터 수집하기

- data와 target의 크기를 확인해 보면 특성인 data는 442행 10열로 이루어져 있고, 타깃인 target은 442개로 구성되어 있음

```
01 print(diabetes['data'].shape, diabetes['target'].shape)
```

```
(442, 10) (442,)
```

4. [플러스 예제] 당뇨병 진행도 예측

■ 데이터 전처리하기

- 사이킷런의 `train_test_split` 함수를 이용해서 특성인 `diabetes['data']`와 티깃인 `diabetes['target']`을 훈련 세트인 `X_train`과 `y_train`, 테스트 세트인 `X_test`와 `y_test`로 나눔

```
01 from sklearn.model_selection import train_test_split
02
03 X_train, X_test, y_train, y_test = train_test_split(diabetes['data'],\
04                                                    diabetes['target'], random_state=0)
05 print(X_train.shape, y_train.shape)
06 print(X_test.shape, y_test.shape)
```

```
(331, 10) (331,)
```

```
(111, 10) (111,)
```

4. [플러스 예제] 당뇨병 진행도 예측

■ 데이터 전처리하기

- 특성 값의 범위가 유사하게 되도록 데이터 스케일링을 훈련 세트의 특성과 테스트 세트의 특성에 대해 진행

```
01 from sklearn.preprocessing import StandardScaler
02
03 scaler = StandardScaler()
04 X_train = scaler.fit_transform(X_train)
05 X_test = scaler.transform(X_test)
```

4. [플러스 예제] 당뇨병 진행도 예측

■ 모델 생성하기

① 모델 정의하기

- Sequential 함수를 이용해서 순차적인 모델인 model을 생성한 후에 add 함수를 이용해서 3개의 Dense 층을 추가

```
01 from tensorflow.keras.models import Sequential
02 from tensorflow.keras.layers import Dense
03
04 model = Sequential()
05 model.add(Dense(16, activation='relu', input_dim=10))
06 model.add(Dense(16, activation='relu'))
07 model.add(Dense(1))
```

4. [플러스 예제] 당뇨병 진행도 예측

■ 모델 생성하기

① 모델 정의하기

- 마지막 층이 출력층이고 나머지 두 개의 층이 은닉층
- 은닉층과 입력층 역할을 겸하고 있는 첫 번째 층에서는 입력의 수를 샘플의 특성 개수와 같은 10으로 설정
- 첫 번째 층과 두 번째 층의 활성화 함수는 relu로 설정했으며, 노드 수는 임의로 16을 설정
- 회귀 문제이므로 출력층에서 활성화 함수를 지정하지 않고 노드 수는 1로 설정

4. [플러스 예제] 당뇨병 진행도 예측

■ 모델 생성하기

① 모델 정의하기

```
model.add(Dense(16, activation='relu', input_dim=10))
model.add(Dense(16, activation='relu'))
model.add(Dense(1))
```

노드 수 설정 입력 수 설정

입력층, 은닉층 은닉층 출력층

4. [플러스 예제] 당뇨병 진행도 예측

■ 모델 생성하기

② 모델 컴파일하기

- 최적화 함수를 adam으로 설정하고, 회귀 문제이므로 손실 함수를 mse로 설정
- 측정 지표는 mae로 설정

```
01 model.compile(optimizer='adam', loss='mse', metrics=['mae'])
```


4. [플러스 예제] 당뇨병 진행도 예측

■ 모델 생성하기

③ 모델 학습시키기

- 특성인 `X_train`과 타깃인 `y_train`을 이용해서 `model`을 학습시킴
- 학습 횟수는 50, 배치 사이즈는 2로 설정하고 테스트 세트를 검증 데이터로 설정
- `fit` 함수의 실행 결과는 `history`에 저장

4. [플러스 예제] 당뇨병 진행도 예측

■ 모델 생성하기

③ 모델 학습시키기

```
01 history = model.fit(X_train, y_train, epochs=50, batch_size=2,\n02                       validation_data=(X_test, y_test))
```

```
Epoch 1/50\n166/166 [=====] - 2s 8ms/step - loss: 29133.1172 - mae: 151.2266 -\nval_loss: 27637.2402 - val_mae: 150.4935\nEpoch 2/50\n166/166 [=====] - 1s 5ms/step - loss: 27262.6426 - mae: 144.9923 -\nval_loss: 24128.3242 - val_mae: 138.6179\nEpoch 3/50\n166/166 [=====] - 1s 5ms/step - loss: 20696.2676 - mae: 121.8849 -\nval_loss: 15226.0869 - val_mae: 104.6811\n\n:\nEpoch 49/50\n166/166 [=====] - 1s 4ms/step - loss: 2415.4656 - mae: 39.5663 -\nval_loss: 3316.5283 - val_mae: 45.2383\nEpoch 50/50\n166/166 [=====] - 1s 4ms/step - loss: 2407.4968 - mae: 39.3935 -\nval_loss: 3321.9841 - val_mae: 45.2593
```

4. [플러스 예제] 당뇨병 진행도 예측

■ 모델 평가하기

- 모델에 대한 훈련 세트의 성능과 테스트 세트의 성능을 평가하는 코드는 다음과 같음

```
01 print(model.evaluate(X_train, y_train))  
02 print(model.evaluate(X_test, y_test))
```

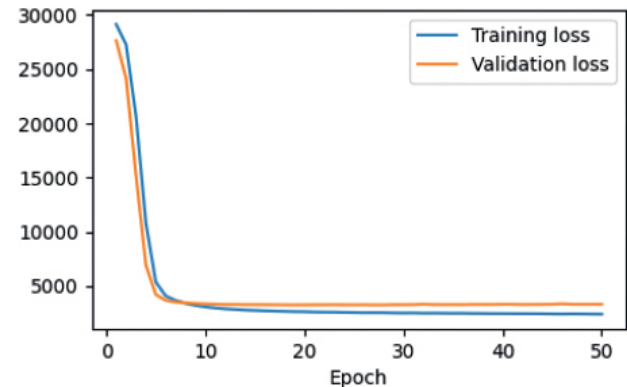
```
11/11 [=====] - 0s 2ms/step - loss: 2366.7761 - mae: 39.0974  
[2366.776123046875, 39.09744644165039]  
4/4 [=====] - 0s 4ms/step - loss: 3321.9841 - mae: 45.2593  
[3321.984130859375, 45.25925064086914]
```

4. [플러스 예제] 당뇨병 진행도 예측

■ 모델 평가하기

- 매 반복에서 얻은 훈련 세트와 검증 데이터의 손실값을 그래프로 나타냄
- 훈련 세트에서의 손실값은 `history.history['loss']`, 검증 데이터에서의 손실값은 `history.history['val_loss']`

```
01 import numpy as np
02 import matplotlib.pyplot as plt
03
04 epochs = np.arange(1, len(history.history['loss']) + 1)
05
06 plt.figure(figsize=(5,3))
07 plt.plot(epochs, history.history['loss'], label='Training loss')
08 plt.plot(epochs, history.history['val_loss'], label='Validation loss')
09 plt.xlabel('Epoch')
10 plt.legend()
11 plt.show()
```



4. [플러스 예제] 당뇨병 진행도 예측

■ 모델 평가하기

■ 전체 프로그램

```
01 from sklearn import datasets
02 from sklearn.model_selection import train_test_split
03 from sklearn.preprocessing import StandardScaler
04 from tensorflow.keras.models import Sequential
05 from tensorflow.keras.layers import Dense
06
07 # 데이터 수집하기
08 diabetes = datasets.load_diabetes()
09
10 # 데이터 전처리하기
11 X_train, X_test, y_train, y_test = train_test_split(diabetes['data'],\
12                                                     diabetes['target'], random_state=0)
13 print(X_train.shape, y_train.shape)
14 print(X_test.shape, y_test.shape)
15
```

4. [플러스 예제] 당뇨병 진행도 예측

■ 모델 평가하기

■ 전체 프로그램

```
16 scaler = StandardScaler()
17 X_train = scaler.fit_transform(X_train)
18 X_test = scaler.transform(X_test)
19
20 # 모델 생성하기
21 model = Sequential()
22 model.add(Dense(16, activation='relu', input_dim=10))
23 model.add(Dense(16, activation='relu'))
24 model.add(Dense(1))
25
26 model.compile(optimizer='adam', loss='mse', metrics=['mae'])
27
```

4. [플러스 예제] 당뇨병 진행도 예측

■ 모델 평가하기

■ 전체 프로그램

```
28 history = model.fit(X_train, y_train, epochs=50, batch_size=2,\n29                     validation_data=(X_test, y_test))\n30\n31 # 모델 평가하기\n32 print(model.evaluate(X_train, y_train))\n33 print(model.evaluate(X_test, y_test))
```

```
(331, 10) (331,)\n(111, 10) (111,)\nEpoch 1/50\n166/166 [=====] - 2s 8ms/step - loss: 29133.1172 - mae: 151.2266 -\nval_loss: 27637.2402 - val_mae: 150.4935\nEpoch 2/50\n166/166 [=====] - 1s 5ms/step - loss: 27262.6426 - mae: 144.9923 -\nval_loss: 24128.3242 - val_mae: 138.6179\nEpoch 3/50\n166/166 [=====] - 1s 5ms/step - loss: 20696.2676 - mae: 121.8849 -\nval_loss: 15226.0869 - val_mae: 104.6811
```

4. [플러스 예제] 당뇨병 진행도 예측

■ 모델 평가하기

■ 전체 프로그램

```
:  
Epoch 49/50  
166/166 [=====] - 1s 4ms/step - loss: 2415.4656 - mae: 39.5663 -  
val_loss: 3316.5283 - val_mae: 45.2383  
Epoch 50/50  
166/166 [=====] - 1s 4ms/step - loss: 2407.4968 - mae: 39.3935 -  
val_loss: 3321.9841 - val_mae: 45.2593  
11/11 [=====] - 0s 2ms/step - loss: 2366.7761 - mae: 39.0974  
[2366.776123046875, 39.09744644165039]  
4/4 [=====] - 0s 4ms/step - loss: 3321.9841 - mae: 45.2593  
[3321.984130859375, 45.25925064086914]
```